

```
import torch
import torchvision

if torch.cuda.is_available():
    print("Successfully enabled CUDA processing")
    device = torch.device("cuda")
else:
    print("CUDA processing not available. Things will be slow :-(")
    device = torch.device("cpu")
```

CUDA processing not available. Things will be slow :-(")

Dataset

In PyTorch, a dataset stores a list of input and target tensors ($X^{[n]}, T^{[n]}$). We will make use of EMNIST dataset for this assignment. In particular, we will use the letters to train the CNN for classification task and then fine-tune this CNN using the digits.

In the **EMNIST** dataset, the inputs are $X^{[n]} \in \mathbb{R}^{28 \times 28}$. In case of the `letters` split, the labels are $T^{[n]} \in \{1, \dots, 26\}$. For `digits` split, the labels $T^{[n]} \in \{0, \dots, 9\}$ correspond to the digit.

More precisely, the data in the dataset is provided in the form of `PIL.Image.Image`, which represents an image class with some more functionality, and pixel values in range `[0, 255]`. To convert these images into `torch.Tensor`'s in range `[0, 1]`, we can use the `torchvision.transforms.ToTensor` transform. Furthermore, in `PyTorch` batches are created from datasets using the `torch.utils.data.DataLoader` class.

Task 2.1: Dataset Loading

Here, we use the letters (`split="letters"`) in EMNIST dataset of gray images for categorical classification, and digits (`split="digits"`) for transfer learning.

Write a function that returns the training and the validation set of the dataset, using the given `transform` and `split`.

Note:

Targets for `letters` range `[1, 26]` by default, which will cause problem when using the loss desired function (which accepts `[0, 25]` instead) in `PyTorch`. Set `target_transform` to a function that can shift the target by subtracting 1 when `split="letters"`.

```
def datasets(split, transform):
    """Instantiates and returns the training and validation sets for the EMNIST dataset for the given split.

    Args:
        split: "letters" or "digits" to obtain the EMNIST letters or the EMNIST MNIST dataset
        transform: The transform to apply to all input images

    Returns:
        trainset: The training partition of the requested dataset
        validset: The validation (aka the test split) of the requested dataset
    """

    if split == "letters":
        # apply lambda function to change the range of targets from [1,26] to [0,25]
        target_transform = lambda x: x - 1
    else:
        target_transform = None

    trainset = torchvision.datasets.EMNIST(root="./data", split=split, train=True, download=True, transform=transform, target_transform=target_transform)
    validset = torchvision.datasets.EMNIST(root="./data", split=split, train=False, download=True, transform=transform, target_transform=target_transform)

    return trainset, validset
```

Test 1: Data Types

Create the dataset with `transform=None`. Check that all inputs are of type `PIL.Image.Image`, and all targets are integral.

```
import PIL
splits_ = ["letters", "digits"]
for split_ in splits_:
    ts_, vs_ = datasets(split=split_, transform=None)

    for x_, t_ in ts_:
        # check datatype of input x
        assert isinstance(x_, PIL.Image.Image)
```

```
# check datatype of target t
assert isinstance(t_, int)
```

100%|██████████| 562M/562M [00:04<00:00, 120MB/s]

Task 2.2: Data Visualization

Create the dataset with `transform=None`. Use `matplotlib` to make a plot with 4 rows and 10 columns.

Since all images in EMNIST dataset have been encoded with mixed-up horizontal and vertical axes, we want to plot the original image, as well as the version with a fixed orientation.

Specifically, in the first row plot 10 images of letter trainset, and in the second row, plot them again with correct orientation. In the third row plot 10 images of digit trainset, and in the fourth row, plot them again with correct orientation.

```
# sometimes the kernel crashes when using matplotlib and pytorch together, the following solves this
import os
os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"

import matplotlib.pyplot
import numpy
matplotlib.pyplot.rcParams['image.cmap'] = 'gray'

# Load dataset without transformation (to show original images)
train_letters, _ = datasets(split="letters",transform=None)
train_digits, _ = datasets(split="digits",transform=None)

# Function to fix exchanged horizontal and vertical axes
def fix_orientation(img):
    return numpy.array(img).T

# Visualization
fig, axes = matplotlib.pyplot.subplots(4, 10, figsize=(15, 6))

# Select 10 samples
for j in range(10):
    # Load letter
    img_letter, _ = train_letters[j]
    fixed_letter = fix_orientation(img_letter)

    # Load digit
    img_digit, _ = train_digits[j]
    fixed_digit = fix_orientation(img_digit)

    # Plot raw and fixed letters
    axes[0, j].imshow(img_letter)
    axes[1, j].imshow(fixed_letter)

    # Plot raw and fixed digits
    axes[2, j].imshow(img_digit)
    axes[3, j].imshow(fixed_digit)

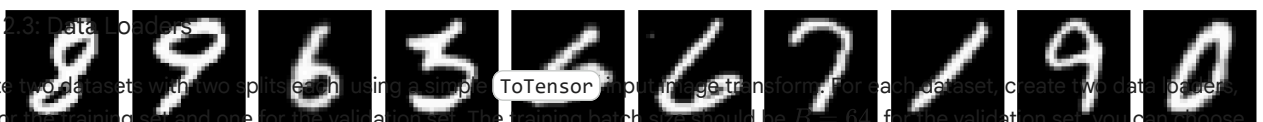
    # Remove axes to show only images
    for i in range(4):
        axes[i, j].axis("off")

matplotlib.pyplot.tight_layout()
matplotlib.pyplot.show()
```



Task 2.3: Data Loaders

Create two datasets with two splits each using a simple `ToTensor` input image transform. For each dataset, create two data loaders, one for the training set and one for the validation set. The training batch size should be $B = 64$, for the validation set, you can choose



any batch size of your choice.

```
# select appropriate transform
transform = torchvision.transforms.ToTensor()

# create datasets
trainset_letters, validset_letters = datasets(split="letters", transform=transform)
trainset_digits, validset_digits = datasets(split="digits", transform=transform)

# instantiate data loaders
trainloader_letters = torch.utils.data.DataLoader(trainset_letters, batch_size=64, shuffle=True)
validloader_letters = torch.utils.data.DataLoader(validset_letters, batch_size=256, shuffle=False)

trainloader_digits = torch.utils.data.DataLoader(trainset_digits, batch_size=64, shuffle=True)
validloader_digits = torch.utils.data.DataLoader(validset_digits, batch_size=256, shuffle=False)
```

✓ Test 2: Batches

Check that all inputs and targets are of type `torch.Tensor`.

Check that all input values are in range $[0, 1]$.

Check that all target values are in range $[0, 25]$ for letters and $[0, 9]$ for digits.

```
for x_, t_ in trainloader_letters:
    # check datatype, size and content of x
    assert isinstance(x_, torch.Tensor)
    assert torch.all(x_ >= 0) and torch.all(x_ <= 1)
    # check datatype, size and content of t
    assert isinstance(t_, torch.Tensor)
    assert torch.all(t_ >= 0) and torch.all(t_ <= 25)

for x_, t_ in trainloader_digits:
    # check datatype, size and content of x
    assert isinstance(x_, torch.Tensor)
    assert torch.all(x_ >= 0) and torch.all(x_ <= 1)

    # check datatype, size and content of t
    assert isinstance(t_, torch.Tensor)
    assert torch.all(t_ >= 0) and torch.all(t_ <= 9)
```

✓ Convolutional Network Training

For training and evaluating the network, we will rely on standard functionality in PyTorch. We will use the standard categorical cross-entropy loss together with a stochastic gradient descent optimizer. For training, we will use the batched implementation of the dataset, for which we perform one update step for each training batch.

For each epoch, we will compute and save the average loss and accuracy for the full training set and validation set. This will be used to visualize the training process of CNN.

✓ Task 2.4: Training and Validation Loop

Implement a function that takes the network, train loader, validation loader, the number of epochs, the learning rate, and the momentum. Select the correct loss function for categorical classification and SGD optimizer. Iterate the following steps for the given number of epochs:

1. Train the network with all batches of the training data.
2. Compute the train set loss, train set accuracy, validation set loss, validation set accuracy.
3. Store all in lists, for later visualization of CNN training process.

Finally, return the lists of train losses and accuracies, as well as validation losses and accuracies.

Notes:

- Make sure that you train on the training data only, and **not** on the validation data.
- When you iterate over validation set, please use `with torch.no_grad():` and loop on validloader inside it. This disables gradient computation and therefore saves memory.
- While saving loss values, please use `.item()`.
- Make sure that you divide the summed loss and accuracy values by the correct count.

```

def training_loop(network, trainloader, validloader, epochs, lr, momentum):
    """Trains the given network for the given number of epochs on the given training data using the given learning rate and
    After each epoch, the validation loss and accuracy are computed using the provided validation data

    Args:
    network: A convolutional network with the correct number of output nodes
    trainloader: The data loader of the training set
    validloader: The data loader for the validation set
    epochs: The number of epochs to train for
    lr: The learning rate to apply
    momentum: the momentum parameter for the learning

    Returns:
    train_loss_list: The average training losses computed during the epochs
    train_acc_list: The average training accuracies computed during the epochs
    val_loss_list: The average validation losses computed after the epochs
    val_acc_list: The average validation accuracies computed after the epochs
    """

    # move the network to the selected device
    network.to(device)

    # select loss function and optimizer
    loss_function = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(network.parameters(), lr=lr, momentum=momentum)

    # collect loss values and accuracies over the training epochs
    train_loss_list, train_acc_list = [], []
    val_loss_list, val_acc_list = [], []

    for epoch in range(epochs):
        network.train()
        # train network on training data
        train_loss, train_correct, train_total = 0, 0, 0
        for x, t in trainloader:
            # put data to device
            x, t = x.to(device), t.to(device)
            # train
            optimizer.zero_grad()
            output = network(x)
            loss = loss_function(output, t)
            # weight update
            loss.backward()
            optimizer.step()
            # calculate training accuracies and losses for current batch
            train_loss += loss.item()
            _, predicted = torch.max(output.data, 1)
            train_total += t.size(0)
            train_correct += (predicted == t).sum().item()
        # append training accuracies and losses for current epoch
        train_loss_list.append(train_loss / len(trainloader))
        train_acc_list.append(100 * train_correct / train_total)

        # Set network to evaluation mode
        network.eval()
        # validate network on validation data
        val_loss, val_correct, val_total = 0, 0, 0
        with torch.no_grad():
            for x, t in validloader:
                # put data to device
                x, t = x.to(device), t.to(device)
                # compute validation loss
                output = network(x)
                loss = loss_function(output, t)
                val_loss += loss.item()
                # compute validation accuracy
                _, predicted = torch.max(output.data, 1)
                val_total += t.size(0)
                val_correct += (predicted == t).sum().item()

        # append validation accuracies and losses for current epoch
        val_loss_list.append(val_loss / len(validloader))
        val_acc_list.append(100 * val_correct / val_total)

        # print training loss, accuracy, validation loss, accuracy for current epoch
        print(f"Epoch {epoch+1}/{epochs}: Train Loss: {train_loss_list[-1]:.4f}, Train Acc: {train_acc_list[-1]:.2f}%, Val

    # return the collected lists
    return train_loss_list, train_acc_list, val_loss_list, val_acc_list

```

We will rely on `torch.nn.Sequential` to create networks with particular lists of consecutive layers.

Implement a function that generates a convolutional network with the following layers:

1. 2D convolutional layer with Q_1 channels, kernel size 7×7 , stride 1 and padding 0.
2. 2D maximum pooling with pooling size 3×3 and stride 3
3. `tanh` activation
4. 2D convolutional layer with Q_2 channels, kernel size 5×5 , stride 1 and padding 2.
5. 2D maximum pooling with pooling size 2×2 and stride 2
6. `tanh` activation
7. A flattening layer to turn the 3D feature map into a 1D vector
8. A fully-connected layer with the appropriate number of inputs and O outputs.

```
def convolutional(Q1, Q2, O):
    """Instantiates a convolutional network used for categorical classification.

    Args:
        Q1: The number of output channels of the first convolutional layer
        Q2: The number of output channels of the second convolutional layer
        O: The number of output logits of the final fully-connected layer

    Returns:
        An instance of a convolutional network with the given parameters
    """
    return torch.nn.Sequential(
        torch.nn.Conv2d(in_channels=1, out_channels=Q1, kernel_size=7, stride=1, padding=0),
        torch.nn.MaxPool2d(kernel_size=3, stride=3),
        torch.nn.Tanh(),
        torch.nn.Conv2d(in_channels=Q1, out_channels=Q2, kernel_size=5, stride=1, padding=2),
        torch.nn.MaxPool2d(kernel_size=2, stride=2),
        torch.nn.Tanh(),
        torch.nn.Flatten(),
        torch.nn.Linear(in_features=Q2 * 3 * 3, out_features=O)
    )
```

✓ Test 3: Network Implementation

Create a network with an arbitrary shape.

Create a batch that follows input dimensions.

Forward the batch through the network.

Check that the output dimensions fit.

```
net_ = convolutional(3,4,6)
batch_ = torch.rand((8,1,28,28))
output_ = net_(batch_)
assert output_.shape == (8,6)
```

✓ Task 2.6: Convolutional Training

Create a convolutional network with $Q_1 = 16$ and $Q_2 = 32$ convolutional channels and $O = 26$ output neurons. Train the network for 5 epochs with $\eta = 0.01$, momentum = 0.9 and store the obtained train losses, accuracies, test losses and accuracies.

If you want, you can also train for 20 epochs, the training time will increase accordingly -- it might take up to 30 minutes on the CPU.

```
# instantiate initial network according to the above description
initial_network = convolutional(Q1=16, Q2=32, O=26)
# train the network on the Letters training data
train_loss, train_acc, val_loss, val_acc = training_loop(initial_network, trainloader_letters, validloader_letters, epochs=5)

Epoch 1/5: Train Loss: 0.7383, Train Acc: 78.66%, Val Loss: 0.3700, Val Acc: 88.88%
Epoch 2/5: Train Loss: 0.3233, Train Acc: 89.83%, Val Loss: 0.3065, Val Acc: 90.29%
Epoch 3/5: Train Loss: 0.2719, Train Acc: 91.35%, Val Loss: 0.2722, Val Acc: 91.46%
Epoch 4/5: Train Loss: 0.2455, Train Acc: 92.05%, Val Loss: 0.2592, Val Acc: 91.68%
Epoch 5/5: Train Loss: 0.2276, Train Acc: 92.59%, Val Loss: 0.2472, Val Acc: 92.07%
```

✓ Task 2.7: Plotting

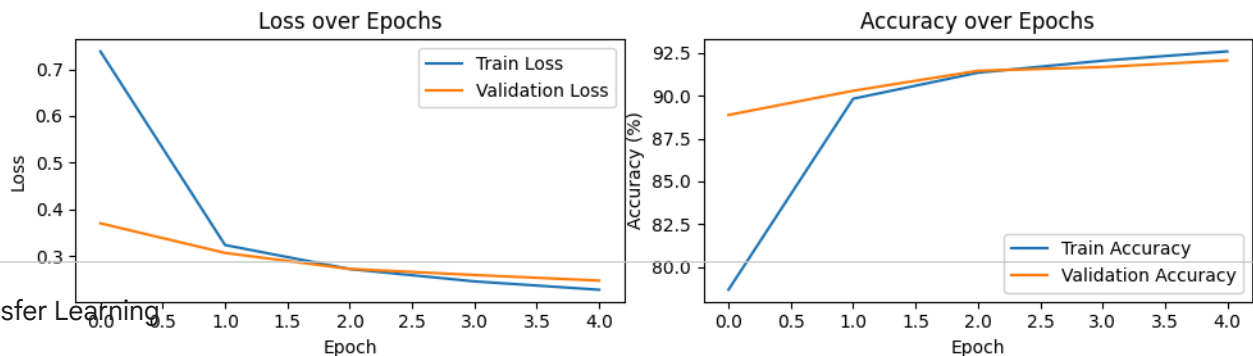
Plot the loss values in one plot and accuracy values into another plot.

```
matplotlib.pyplot.figure(figsize=(10,3))
ax = matplotlib.pyplot.subplot(121)
```

```
# plot training and validation loss values of network over epochs
ax.plot(train_loss, label='Train Loss')
ax.plot(val_loss, label='Validation Loss')
ax.set_title('Loss over Epochs')
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')
ax.legend()

ax = matplotlib.pyplot.subplot(122)
# plot training and validation accuracy values of network over epochs
ax.plot(train_acc, label='Train Accuracy')
ax.plot(val_acc, label='Validation Accuracy')
ax.set_title('Accuracy over Epochs')
ax.set_xlabel('Epoch')
ax.set_ylabel('Accuracy (%)')
ax.legend()

matplotlib.pyplot.tight_layout()
matplotlib.pyplot.show()
```



Transfer Learning

We will make use of above trained CNN, which can be used to classify 26 handwritten characters, and fine-tune this CNN as to the digits classification task instead.

Task 2.8: Pre-trained Network Instantiation

Make a copy of the trained CNN.

Freeze the feature layers of the copied network.

Notes:

- To freeze layers, you can simply disable gradient computation for all learnable parameters of the network via `parameter.requires_grad = False`.

```
import copy
# copy trained convolutional network
network_copy = copy.deepcopy(initial_network).to("cpu")

# Make sure to freeze all the layers of the network.
for param in network_copy.parameters():
    param.requires_grad = False
```

Task 2.9: Network Implementation

We want to modify the network such that we extract the logits for the 10 classes from the last fully-connected layer of the network.

Implement a function that replaces the current last linear layer of the pre-trained network with a new linear layer that has O units (O represents the number of classes in our dataset).

```
def replace_last_layer(network, O=10):
    """Replaces the final layer with the given number of output units, and reinitializes the weights

    Args:
        network: A convolutional network, where the last layer is a fully-connected layer
        O: The number of output neurons for the new fully-connected layer

    Returns:
        network: The input network with the last layer replaced
    """

    # get the number of input features from the current last layer
    in_features = network[-1].in_features
    # create a new linear layer with the desired number of output units
    new_last_layer = torch.nn.Linear(in_features, O)
```

```
# replace the last linear layer of the network with the new layer
network[-1] = new_last_layer

return network
```

✓ Test 4: Last Layer Dimensions

This test ensures that the function return a network having the correct number of input and output units in the last layer.

```
O_ = 6

test_network_ = replace_last_layer(network_copy, O=O_)
assert test_network_[-1].out_features == O_
assert test_network_[-1].in_features == 288
```

✓ Task 2.10: Network Fine-Tuning with Frozen Layers

Create a network that has feature layers frozen with 10 output units. Fine-tune the created network for 2 epochs on our digits data (`trainloader_digits`, `validloader_digits`) using the previous function, and a smaller learning rate of $\eta = 0.001$.

```
# replace network layer
network_with_frozen_layers = replace_last_layer(network_copy, O=10)
# fine-tune last layer with the digits data
_ = training_loop(network_with_frozen_layers, trainloader_digits, validloader_digits, epochs=2, lr=0.001, momentum=0.9)
```

Epoch 1/2: Train Loss: 0.1571, Train Acc: 96.78%, Val Loss: 0.0790, Val Acc: 98.09%
Epoch 2/2: Train Loss: 0.0692, Train Acc: 98.25%, Val Loss: 0.0617, Val Acc: 98.40%

✓ Test 5: Frozen Layers

Check that all layers of the fine-tuned network that contain weights and biases (except for the last fully-connected layer) have not been modified by the training, and that the final layer differs in shape.

```
for i in range(len(initial_network)-1):
    if hasattr(initial_network[i], "weight"):
        assert torch.allclose(initial_network[i].weight, network_with_frozen_layers[i].weight)
    if hasattr(initial_network[i], "bias"):
        assert torch.allclose(initial_network[i].bias, network_with_frozen_layers[i].bias)

assert initial_network[-1].weight.shape != network_with_frozen_layers[-1].weight.shape
assert initial_network[-1].bias.shape != network_with_frozen_layers[-1].bias.shape
```

✓ Task 2.11: Test Set Predictions

Go over the validation set of the Digits dataset once again and extract predictions of the network.

```
predictions = []
targets = []
with torch.no_grad():
    for x, t in validloader_digits:
        # put data onto device
        x, t = x.to(device), t.to(device)
        # extract logits from the network
        output = network_with_frozen_layers(x)
        # obtain predicted class
        _, predicted_class = torch.max(output.data, 1)

        # store predicted class
        predictions.extend(predicted_class.cpu().tolist())

        # store target class
        targets.extend(t.tolist())
```

✓ Task 2.12: Confusion Matrix Plotting

Finally, we want to plot the confusion matrix of the validation set. For this, we can make use of the

`sklearn.metrics.confusion_matrix` to compute the confusion matrix. You can utilize

`sklearn.metrics.ConfusionMatrixDisplay` for displaying the confusion matrix, or `pyplot.imshow` and adding the according labels.

Note:

- The documentation for the confusion matrix can be found here: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html
- The interface and an example for the `ConfusionMatrixDisplay` can be found here: <https://scikit-learn.org/stable/modules/generated/sklearn.metrics.ConfusionMatrixDisplay.html>

```
import sklearn.metrics

# get the target class names
classes = trainset_digits.classes

# compute confusion matrix
matrix = sklearn.metrics.confusion_matrix(targets, predictions)

# plot confusion matrices
plot_conf_matrix1 = sklearn.metrics.ConfusionMatrixDisplay(matrix, display_labels=classes)
plot_conf_matrix1.plot(xticks_rotation="vertical")
```

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7dceb8ad4680>

